Intelligent and Converged Networks ISSN 2708-6240
2024, 5(2): 167−179 DOI: 10.23919/ICN.2024.0012

Edge-assisted indexing for highly dynamic and static data in mixed reality connected autonomous vehicles

Daniel Mawunyo Doe*, Dawei Chen, Kyungtae Han, Haoxin Wang, Jiang Xie, and Zhu Han

Abstract: The integration of Mixed Reality (MR) technology into Autonomous Vehicles (AVs) has ushered in a new era for the automotive industry, offering heightened safety, convenience, and passenger comfort. However, the substantial and varied data generated by MR-Connected AVs (MR-CAVs), encompassing both highly dynamic and static information, presents formidable challenges for efficient data management and retrieval. In this paper, we formulate our indexing problem as a constrained optimization problem, with the aim of maximizing the utility function that represents the overall performance of our indexing system. This optimization problem encompasses multiple decision variables and constraints, rendering it mathematically infeasible to solve directly. Therefore, we propose a heuristic algorithm to address the combinatorial complexity of the problem. Our heuristic indexing algorithm efficiently divides data into highly dynamic and static categories, distributing the index across Roadside Units (RSUs) and optimizing query processing. Our approach takes advantage of the computational capabilities of edge servers or RSUs to perform indexing operations, thereby shifting the burden away from the vehicles themselves. Our algorithm strategically places data in the cache, optimizing cache hit rate and space utilization while reducing latency. The quantitative evaluation demonstrates the superiority of our proposed scheme, with significant reductions in latency (averaging 27%−49.25%), a 30.75% improvement in throughput, a 22.50% enhancement in cache hit rate, and a 32%−50.75% improvement in space utilization compared to baseline schemes.

Key words: mixed reality; autonomous vehicles; data indexing; edge computing; query optimization

1 Introduction

The fusion of Mixed Reality (MR) technology with Autonomous Vehicles (AVs) has brought about a profound transformation in the automotive industry, enhancing safety, convenience, and passenger comfort. Nonetheless, this advancement comes with the challenge of managing an extensive array of data, encompassing both highly dynamic and static information, generated by MR-Connected AVs (MR-CAVs). The necessity for real-time data retrieval and processing in the ever-evolving vehicular environments underscores the importance of devising an optimal indexing system[1]. Such a system must proficiently handle the diverse data spectrum to guarantee the secure and efficient functioning of MR-CAVs.

Highly dynamic data in the vehicular context refer to real-time information continuously produced by sensors, cameras, and other onboard devices in MR-CAVs. Examples of highly dynamic data encompass traffic conditions, weather updates, location details, and vehicle telemetry. On the other hand, static data

• Daniel Mawunyo Doe and Zhu Han are with the Department of Electrical and Computer Engineering, University of Houston, Houston, TX 77004, USA. E-mail: dmdoe@uh.edu; hanzhu22@gmail.com.
• Dawei Chen and Kyungtae Han are with InfoTech Labs, Toyota Motor North America Research and Development, Mountain View, CA 94043, USA. E-mail: dawei.chen1@toyota.com; kyungtae.han@toyota.com.
• Haoxin Wang is with the Department of Computer Science, Georgia State University, Atlanta, GA 30302, USA. E-mail: haoxinwang@gsu.edu.
• Jiang Xie is with the Department of Electrical and Computer Engineering, University of North Carolina at Charlotte, Charlotte, NC 28223, USA. E-mail: jxie1@uncc.edu.
* To whom correspondence should be addressed.
Manuscript received: 2023-12-25; revised: 2024-02-15; accepted: 2024-03-25

comprise relatively stable information that undergoes infrequent changes, such as road maps, traffic regulations, and points of interest. Both highly dynamic and static data are indispensable for the operation of MR-CAVs, providing vital insights for navigation, safety, and the overall driving experience[1]. While highly dynamic data enables AVs to make real-time decisions and adapt to changing road conditions, static data form the foundation for safe and efficient operation within a specific environment[2].

As the volume of generated data continues to increase, conventional storage and retrieval methods may become inefficient and slow, leading to increased latency and reduced performance. The primary challenge lies in managing the voluminous and heterogeneous data generated by these vehicles and their surroundings, requiring an indexing solution that can adapt to the high velocity and variability of data. Therefore, we need an efficient data indexing system that can optimize data retrieval by organizing the data and providing quick access to the necessary data elements. Several studies have been conducted on data indexing for vehicular networks. For example, the authors of Refs. [3, 4] proposed Content Delivery Networks (CDNs) to improve data delivery and enhance user experience in vehicular networks. However, CDNs are primarily designed for static content delivery and may struggle to handle the highly dynamic data generated in real-time by MR-CAVs. Also, CDNs rely on centralized data centers, which can introduce significant latency due to the long-distance communication between vehicles and the data centers. The authors of Refs. [5, 6] proposed an approach that uses edge servers to index and store vehicle-generated data. However, their system suffers from high latency and limited scalability. Similarly, the authors of Refs. [7, 8] developed a caching-based approach for indexing and retrieving data, but their process is limited in terms of the types of data that can be indexed and retrieved.

In Refs. [9, 10], machine learning based approaches were proposed to predict future data access. However, they have limitations: Ref. [9] focuses on driver state awareness for handover scenarios, not addressing broader data management challenges in MR-CAVs, while Ref. [10] primarily optimizes content delivery in vehicle networks, overlooking the specific challenges of managing diverse data in MR-CAVs. Motivated by these limitations and the need for efficient data management in MR-CAVs, our paper addresses these key research questions:

(1) How can an algorithm efficiently categorize and index both highly dynamic and static data in MR-CAVs?

(2) What strategies can optimize query processing and data retrieval in MR-CAVs?

(3) How does our proposed algorithm outperform existing methods in indexing data for MR-CAVs?

To address the limitations of the existing approaches, we propose an algorithm that handles highly dynamic and static data on Roadside Units (RSUs), enabling efficient data retrieval by MR-CAVs. Our approach presents several key contributions: firstly, it adeptly manages both highly dynamic and static data, ensuring real-time updates for dynamic data and periodic updates for static data. Secondly, it incorporates a query optimization algorithm to boost performance and reduce latency. Lastly, our algorithm offers a comprehensive and scalable solution for indexing and retrieving highly dynamic and static data on RSUs. Our optimization problem formulation, detailed in Section 4.1, casts the challenge as a constrained optimization problem aiming to maximize a utility function representing the overall performance of our indexing system. However, due to the combinatorial complexity stemming from numerous decision variables and constraints, direct mathematical solutions are infeasible. Additionally, the problem's complexity escalates with factors like the number of RSUs, dataset size, and data update frequency, while the non-convex nature of the objective function and non-linear constraints further complicates the search for a unique or globally optimal solution. Therefore, we propose employing a heuristic algorithm to tackle this indexing problem, allowing us to efficiently explore the solution space and discover suitable approximations to the optimal solution. Our heuristic indexing algorithm, described in Section 4.2, solves the optimization problem through a series of steps, including index

initialization, distribution across RSUs, caching, and query optimization, ultimately providing efficient data retrieval by MR-CAVs. The main contributions of this work are summarized as follows:

- We propose an algorithm that divides data into highly dynamic and static categories, creating specialized indexing systems for each category to optimize data retrieval.
- We design a query optimization algorithm to ensure high availability, low latency, and improved performance for efficient data retrieval in vehicular environments.
- We conduct extensive experiments to demonstrate the effectiveness of our proposed indexing scheme for highly dynamic and static data in MR-CAVs.

The paper's structure is outlined as follows: Section 2 provides an overview of related works, Section 3 presents the system model, Section 4 introduces our edge-assisted indexing approach for highly dynamic and static data, Section 5 analyzes the simulation results, and Section 6 summarizes our findings.

2 Related work

The rapid development of MR-CAVs has brought forth a wave of research in data management, particularly concerning edge computing and indexing strategies. Several works explore the potential of edge computing in alleviating computational burdens on AVs, with a general focus on vehicular networks without delving deep into the intricacies of MR environments (e.g., Refs. [2, 11]). In the realm of data indexing, dynamic data indexing methods and reviews of various techniques for vehicular networks have been presented in Refs. [12, 13]. However, these approaches often lack comprehensiveness, either focusing on urban environments or broader vehicular networks without addressing the unique data challenges posed by MR technologies.

Recognizing the significance of query optimization and caching strategies in MR-enabled AVs, existing literature attempts to address these aspects but often falls short of holistically meeting the systems' needs. Research by Refs. [14, 15] optimizes data queries for AR and VR applications, respectively, yet neglects the combined requirements of these environments, often emphasizing one aspect while overlooking others. Similarly, caching strategies like those proposed in Refs. [16, 17] introduce solutions for high-speed vehicular networks and edge networks but fail to account for the variable and complex nature of MR-connected AV data.

As the volume of data produced increases, conventional methods for storing and retrieving data are becoming less efficient, often resulting in heightened latency and diminished performance. Research in the field of data indexing for vehicular networks has been extensive. Studies such as those by Refs. [3, 4] have advocated for the use of CDNs to enhance data distribution and user experience within these networks. Yet, CDNs are typically more suited for delivering static content and may falter in managing the dynamic data generated by MR-CAVs in real-time. A significant drawback of CDNs is their dependence on centralized data centers, potentially leading to increased latency due to the extended communication distance between the vehicles and these centers. In a similar scope, Refs. [5, 6] suggested utilizing edge servers for the indexing and storage of data generated by vehicles. However, this approach is hampered by issues such as high latency and scalability constraints. Further, Refs. [7, 8] proposed a caching-based methodology for data indexing and retrieval. Nevertheless, this method is limited in its ability to index and retrieve diverse types of data. Additionally, approaches employing machine learning techniques for future data access predictions, as proposed by Refs. [9, 10], are also limited by the predictive accuracy of their models.

This gap in existing research highlights the need for a comprehensive approach that tackles both highly dynamic and static data challenges specific to MR-enabled AVs. Our work bridges this gap by proposing a novel indexing algorithm that leverages edge computing to optimize data retrieval and minimize latency. We uniquely focus on the intricacies of managing both data types in MR environments, an area not extensively explored in current research. By addressing these shortcomings, our algorithm presents

a significant advancement in the field, offering a tailored solution for efficient and effective data management in MR-CAVs.

3 System model

This section presents the system model, which comprises various components essential for our proposed data indexing architecture in MR-CAVs. In Section 3.1, we introduce the architecture that integrates MR headsets with RSUs. The network model in Section 3.2 outlines the interactions between MR-CAVs and RSUs. Finally, Section 3.3 describes a utility function that evaluates system performance.

3.1 System overview

Our system architecture combines MR headsets and RSUs, as illustrated in Fig. 1, presenting our indexing system for highly dynamic and static data. MR headsets serve as user interfaces, enabling users to search for and retrieve environmental sensing data. RSUs act as intermediaries, managing indexed and cached data to ensure accessibility from MR headsets. Each RSU, co-located with a Multi-access Edge Computing (MEC) server, provides ample computational resources for processing vehicular data in its designated region (target cell). RSUs are equipped with sensors, including high-definition cameras and storage capacity, for processing, collecting, and storing environmental data.

To optimize the indexing and retrieval of highly dynamic and static data, our system utilizes a combination of indexing techniques. We employ a real-time indexing system for highly dynamic data, which frequently updates the index to maintain data accuracy and currency[18]. Conversely, for static data, we implement a one-time indexing system that constructs the index initially and updates it periodically to preserve accuracy. Our query optimization algorithm evaluates incoming queries and selects the most efficient indexing system for data retrieval. By integrating these indexing techniques and enhancing data retrieval performance, our system architecture efficiently manages highly dynamic and static data within vehicular environments.

3.2 Network model

Our network model comprises a population of V MR-CAVs and N number of RSUs. Let $\mathcal{V}$ denote the set of AVs, such that $\mathcal{V}=\{1,2,\ldots,V\}$ and let $\mathcal{N}$ represent the set of RSUs, such that $\mathcal{N}=\{1,2,\ldots,N\}$. We design our model to reflect a realistic vehicular network topology, aligning with the models used in established works such as Refs. [19, 20], which discuss vehicular network dynamics and resource constraints in edge computing environments. The quantity of index data is assumed to be Q bytes, where Q_{H} and Q_{S} represent highly dynamic and static data, respectively. Let $t_{Q_{\mathrm{H}}}$ and $C_{Q_{\mathrm{H}}}$ be the time taken and cost of updating the index for

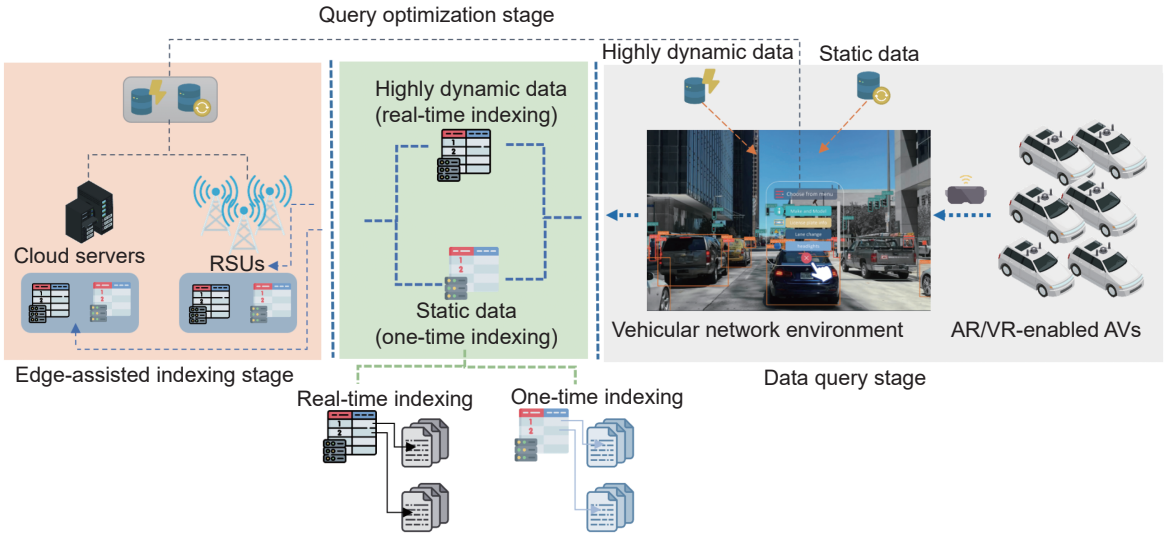

Fig. 1 Our proposed highly dynamic and static data indexing system architecture.

highly dynamic data, respectively. The highly dynamic data indexing system is modeled as a real-time system with continuous updates, represented by

$$t_{Q_{\mathrm{H}}} = \frac{1}{f_{Q_{\mathrm{H}}}} \tag{1}$$

where $f_{Q_{\mathrm{H}}}$ is the frequency of updates for highly dynamic data, which is considered to be time-varying. The cost of updating the index for highly dynamic data is represented by

$$C_{Q_{\mathrm{H}}} = \eta_{Q_{\mathrm{H}}} \times r_{Q_{\mathrm{H}}} \tag{2}$$

where $\eta_{Q_{\mathrm{H}}}$ is the number of updates for highly dynamic data and $r_{Q_{\mathrm{H}}}$ denotes the resource demands, such as CPU and memory, for each update of highly dynamic data[21].

The static data indexing system is modeled as a batch processing system that updates periodically. Let $t_{Q_{\mathrm{S}}}$ and $C_{Q_{\mathrm{S}}}$ be the time taken and cost of updating the index for static data, respectively. $t_{Q_{\mathrm{S}}}$ can be represented by

$$t_{Q_{\mathrm{S}}} = \eta_{Q_{\mathrm{S}}} \times \tau_{Q_{\mathrm{S}}} \tag{3}$$

where $\eta_{Q_{\mathrm{S}}}$ is the number of updates for static data and $\tau_{Q_{\mathrm{S}}}$ is the time taken to perform each update for static data. Similarly, the cost of updating the index for static data is represented by

$$C_{Q_{\mathrm{S}}} = \eta_{Q_{\mathrm{S}}} \times r_{Q_{\mathrm{S}}} \tag{4}$$

where $r_{Q_{\mathrm{S}}}$ represents the resource demands of each update for static data. The cost C_{Q_n} of storing data Q on RSU n, for $n \in \mathcal{N}$ is given by

$$C_{Q_n} = \eta_{Q_n} \times r_{Q_n} \tag{5}$$

where η_{Q_n} is the number of times that data are accessed from the RSU n and r_{Q_n} is the cost in terms of resources used for storing data on the RSU. We mention that the cost models for both dynamic and static data indexing ($C_{Q_{\mathrm{H}}}$ and $C_{Q_{\mathrm{S}}}$) incorporate resource demand considerations, mirroring the frameworks discussed in Ref. [22], which delves into resource allocation strategies in edge computing.

3.3 Utility model

Our utility function is designed to evaluate the overall system performance, which integrates time and cost factors for both highly dynamic and static data, reflecting a comprehensive approach to system optimization. The utility function U is defined as

$$U = (\sigma_{\mathrm{H}} \times t_{Q_{\mathrm{H}}} \times C_{Q_{\mathrm{H}}}) + (\sigma_{\mathrm{S}} \times t_{Q_{\mathrm{S}}} \times C_{Q_{\mathrm{S}}}) + (\sigma_{\mathrm{C}} \times C_{Q_n}) \tag{6}$$

where σ_{H}, σ_{S}, and σ_{C} are weights representing the relative importance of highly dynamic data, static data, and cost, respectively. This model is inspired by the utility theory applied in network optimization tasks, as detailed in our work[21]. The inclusion of weights σ_{H}, σ_{S}, and σ_{C} allows for the customization of the optimization process, accommodating various application scenarios and environmental conditions. This aspect draws from the work presented in Ref. [21], which emphasizes the adaptability of utility functions in network systems.

Maximizing the utility function U enables a comprehensive evaluation of system performance by considering both the time and monetary costs. Rather than focusing solely on cost minimization, this approach aims to find configurations that provide the best overall trade-off between time and monetary considerations. In the subsequent section, we introduce key decision variables x_{H_n}, x_{S_n}, and x_{C_n}, which connect to the utility function and play a pivotal role in optimizing the system's performance for highly dynamic and static data retrieval.

4 Proposed approach: Edge-based indexing for highly dynamic and static data

In Section 4, we delve into the core components of our indexing methodology for MR-CAVs. Section 4.1 outlines the optimization problem. In Section 4.2, we detail our heuristic indexing algorithm, which encompasses the algorithms for highly dynamic data indexing 1 and static data indexing 2. Lastly, the complexity analysis of proposed scheme (Section 4.3) evaluates the time and space complexity of our approach.

4.1 Problem formulation

We formulate our indexing problem as a constrained optimization problem that maximizes the utility function, which represents the overall performance of our indexing system. By maximizing the utility function, we aim to achieve the highest possible value

for the system's performance, which is desirable for efficient data retrieval in MR-CAVs in vehicular networks. The optimization problem can be represented as

$$\max_{x_{\mathrm{H}_n}, x_{\mathrm{S}_n}, x_{\mathrm{C}_n}} \quad U \tag{7}$$

$$\text{s.t.,} \quad \sum_{i=1}^{n} x_{\mathrm{H}_n} \leqslant N_{\mathrm{H}} \tag{8}$$

$$\sum_{i=1}^{n} x_{\mathrm{S}_n} \leqslant \mathrm{N_S} \tag{9}$$

$$\sum_{i=1}^{n} x_{\mathrm{C}_n} \leqslant N_{\mathrm{C}} \tag{10}$$

$$\sum_{i=1}^{n} C_{Q_{\mathrm{H}}} \leqslant B_{\mathrm{H}} \tag{11}$$

$$\sum_{i=1}^{n} C_{Q_{\mathrm{S}}} \leqslant B_{\mathrm{S}} \tag{12}$$

$$\sum_{i=1}^{n} C_{Q_n} \leqslant B_{\mathrm{C}} \tag{13}$$

$$x_{\mathrm{H}_n}, x_{\mathrm{S}_n}, x_{\mathrm{C}_n} \in \{0,1\} \tag{14}$$

$$C_{Q_{\mathrm{H}}}, C_{Q_{\mathrm{S}}}, C_{Q_n} \geqslant 0 \tag{15}$$

In this optimization problem, we introduce binary decision variables x_{H_n}, x_{S_n}, and x_{C_n}, which serve as indicators for the utilization of RSU n for indexing highly dynamic data, static data, and caching, respectively. Constraints in Formulae (8)−(10) prescribe the limitations on the maximum number of RSUs available for each data type (N_{H}, N_{S}, N_{C}). Meanwhile, constraints in Formulae (11)−(13) define the maximum storage capacity constraints associated with each RSU for the respective data types (B_{H}, B_{S}, B_{C}). Constraints in Formulae (14) and (15) impose restrictions on binary variables x_{H_n}, x_{S_n}, and x_{C_n} and establish non-negativity conditions for the storage cost ($C_{Q_{\mathrm{H}}}$, $C_{Q_{\mathrm{S}}}$, C_{Q_n}) associated with each data type.

While attempting to solve this optimization problem mathematically for our indexing algorithm, it becomes evident that feasibility is impeded by the proliferation of decision variables and constraints. The complexity of this problem escalates notably with the growing number of RSUs, dataset size, and data update frequency. Furthermore, the optimization problem's non-convex objective function and the nonlinear nature of its constraints render it unlikely to possess a unique or globally optimal solution. Consequently, we advocate for the utilization of a heuristic algorithm as an alternative approach to address our indexing problem. Heuristic algorithms are tailored to seek satisfactory solutions within a reasonable timeframe, employing techniques like trial-and-error, localized exploration, or problem-specific heuristics. By employing a heuristic algorithm, we can efficiently navigate the solution space and identify suitable solutions that approximate the optimal outcome. Moreover, this heuristic algorithm can be fine-tuned and adapted to accommodate specific characteristics of the problem, such as data distribution, indexing system parameters, and query workloads, thereby enhancing its performance.

4.2 Proposed edge-based indexing for highly dynamic and static data

Our heuristic indexing procedure addresses the optimization challenge through a series of sequential actions. Initially, the algorithm initiates the indexing for both highly dynamic and static data, employing either a one-time or real-time indexing system. Subsequently, the algorithm employs sharding or replication techniques to distribute the index across multiple RSUs, assuring enhanced accessibility and minimized latency. Concurrently, frequently accessed data are cached to ameliorate latency issues and enhance system performance.

The subsequent phase entails the optimization of query processing, wherein the algorithm intelligently selects the most efficient indexing approach for each incoming query. This optimization process entails query analysis to determine whether it pertains to highly dynamic or static data. Following this assessment, the algorithm examines the network-wide index distribution, identifying the RSUs that are most likely to harbor the pertinent data. Further efficiency is achieved by verifying the presence of the requested data in both local and global caches; if found, the data

are expediently retrieved from the cache, reducing latency. The summary of our indexing algorithm is presented in Algorithm 1.

4.2.1 Highly dynamic data indexing

In the following sections, we present the algorithm for our real-time indexing in Algorithm 2 and elaborate on the necessary steps to design real-time indexing for highly dynamic data as follows:

(1) Data ingestion: The system starts by ingesting highly dynamic data from various sources, such as sensors or streaming services.

(2) Data preprocessing: The ingested data undergo preprocessing to cleanse, transform, and format it into a suitable structure for indexing. This step may involve data cleaning, filtering, and feature extraction, depending on the specific requirements of the data.

(3) Index creation: Next, the system creates an initial index based on the preprocessed data. This index creation can be achieved by employing data structures such as B-trees.

(4) Real-time updates: As new data arrive, the system performs incremental updates to the index in real-time. This update involves efficiently incorporating the new data into the existing index structure without rebuilding the entire index. We employ log-structured merge trees to handle the updates efficiently.

(5) Query processing: The system is capable of processing queries in real-time. When a query is received, it leverages the optimized index structure to retrieve the relevant data quickly. We use query optimization, and edge-caching to improve query performance and reduce latency.

(6) Continuous monitoring and maintenance: The system monitors the data sources and indexes for any changes or updates. If necessary, it performs periodic maintenance tasks, such as index reorganization or optimization, to ensure the index remains accurate and efficient over time.

4.2.2 Static data indexing

We introduce the algorithm for our static data indexing in Algorithm 3 and provide further insights into our implementation in Section 5.1. To establish a one-time indexing system for static data, we contemplate employing a batch processing methodology, encompassing the following stages:

(1) Static data collection: Gather all the static data to be indexed.

Algorithm 1 Efficient indexing algorithm for highly dynamic and static data

Data: Data to be indexed
Result: Indexed data
/* Divide data into highly dynamic and static categories */
dynamicData ← Highly dynamic data; staticData ← Static data;
/* Perform real-time indexing for highly dynamic data */
for each data in dynamicData **do**
    RealTimeIndexing(data);
end
/* Perform one-time indexing for static data */
for each data in staticData **do**
    onetimeindexing(data);
end
/* Distribute index across RSUs (by platform) */
DistributedIndexing();
/* Caching frequently accessed data (by platform) */
Caching();
/* Use query optimization to efficiently retrieve data */
QueryOptimization();

Algorithm 2 Highly dynamic data indexing

Input: Real-time data stream
Output: Real-time index
Initialize empty index;
while Data stream is not empty **do**
    Receive new data point;
    Preprocess data;
    Update index with new data;
    Process queries using the index;
end

Algorithm 3 Static data indexing system

Data: Static data
Result: Index
Preprocess data;
Build initial index;
for each update in periodic updates **do**
    Update index;
end
Optimize index structure;
Store index;
QueryProcessing(query);

(2) Data preprocessing: Cleanse and preprocess the static data to ensure its quality and consistency.

(3) Initial index creation: Generate an initial index by processing the preprocessed static data. This step may involve techniques such as tokenization, stemming, and the creation of inverted indexes.

(4) Periodic index updates: Despite the static nature of the data, occasional updates or additions may occur. Regularly update the index to incorporate any changes in the static data.

(5) Index storage: Store the generated index in an appropriate data storage system that offers rapid and dependable access.

(6) Query processing: Develop mechanisms to efficiently process queries utilizing the generated index. This query processing can encompass techniques like query optimization, relevance ranking, and result retrieval.

4.3 Complexity analysis of the proposed approach

Our proposed algorithm exhibits a time complexity of $O(n\log n)$ during the initial index construction, where n denotes the dataset's size. This complexity arises from the utilization of sorting algorithms during the indexing process. However, the incremental indexing process for highly dynamic data and the periodic updates for static data present lower time complexities of $O(k)$ and $O(m)$, respectively, where k and m represent the sizes of the updated data.

The space complexity of our algorithm scales with the dataset's size. Specifically, the index size for highly dynamic data is confined within a sliding window, while the dataset's size governs the index size for static data. Nevertheless, the distributed indexing system and caching mechanism effectively reduce the memory requirements on individual RSUs. In summary, the space complexity of our algorithm can be optimized through the proficient management of the distributed indexing system and caching mechanism.

5 Experimental result and analysis

In this section, we provide an in-depth analysis of our proposed data indexing architecture for MR-CAVs. Section 5.1 outlines the technical setup, and we conduct a comparative evaluation of our approach against established algorithms[23−26] in the subsequent section. We also present key performance indicators, including average latency, throughput, cache hit rate, and space utilization in Section 5.3. Finally, Section 5.4 offers a comprehensive analysis of the experimental results.

5.1 System configuration

Our proposed system configuration encompasses various components crucial for achieving efficient data retrieval in MR-CAVs. To implement our proposed algorithm, we utilize the Python programming language. The hardware resources in our experimental setup consist of a cluster comprising 20 RSUs, each equipped with 8 GB of RAM and 500 GB of storage. Additionally, a central server with 16 cores, 64 GB RAM, and 1 TB of storage is employed to manage and coordinate the operations of the RSUs. The computational core of our system is based on Python environments running on NVIDIA Jetson Xavier platforms with the Ubuntu 18.04 operating system. Each of these systems is outfitted with 32 GB RAM and 512 GB SSD storage. For the augmented reality interface, we employ the Microsoft HoloLens 2 programmed using Unity 2020.3.48f1 and C# language. The AR environment, crafted within the Unity game engine, is designed to provide users with real-time object detection annotations for indexing.

We have chosen a high-speed communication link, such as 5G, as the backbone of our vehicular network architecture. This choice ensures swift data transmission between MR-CAVs and RSUs, reducing latency and enhancing system responsiveness. Furthermore, in our simulations, we combine Simulation of Urban Mobility (SUMO) with Veins and Objective Modular Network Testbed in C++ (OMNeT++) network simulation framework to model both vehicular movements and wireless communications comprehensively[27]. SUMO is used to generate realistic vehicular mobility patterns based on real-world traffic flow data from urban environments. For the wireless communication aspect, we integrate SUMO with Veins and OMNeT++, employing a basic

wireless communication model provided by the framework. This model simulates key aspects of wireless communications such as signal attenuation and interference, essential for evaluating the performance of MR-CAVs in a realistic setting. This approach allows us to accurately capture the dynamics of both vehicular movements and wireless interactions, enhancing the validity and reproducibility of our research findings. The simulation spans a 24-hour period to simulate real-world scenarios. Weight factors reflecting the relative importance of each component in our utility function are set to $[0.4, 0.4, 0.2]$ for $[\sigma_H, \sigma_S, \sigma_C]$, respectively, following extensive testing to optimize system performance. The experimental cache size is set at 1 GB per RSU. Other parameters affecting our proposed algorithm include the frequency of data updates, network size, and the number of queries per unit of time. These parameters can be adjusted to analyze our system's performance under various scenarios.

5.2 Baseline schemes (benchmarks)

- **Basic indexing algorithm:** This simple indexing algorithm involves creating an index of all the data in the network without any optimization or distribution across RSUs[23].
- **Least Recently Used (LRU) caching algorithm:** This algorithm stores the most recently accessed data in the cache and discards the least recently used data to make room for new data[24].
- **Random caching algorithm:** This algorithm randomly stores data in the cache without specific criteria or optimization[25].
- **Hash-based distribution algorithm:** This algorithm uses a hash function to distribute data across RSUs based on the data's key or identifier[26].

5.3 Metrics of performance

Our study primarily focuses on evaluating system performance and efficiency using average performance metrics. These metrics include average latency, average throughput, cache hit rate, and space utilization. By averaging these values over various experimental scenarios, we can effectively measure and compare the performance of our proposed algorithm against baseline schemes. We outline the various evaluation metrics as follows:

- **Average latency (ms):** This is the time taken for a query to be processed and data to be retrieved. A lower latency indicates a faster system performance.
- **Average throughput (megabits per second (Mbps)):** This is the number of queries that can be processed per unit time. A higher throughput indicates a faster system performance.
- **Cache hit rate (%):** This is the percentage of times a requested data is found in the cache. A higher cache hit rate indicates a better caching performance.
- **Space utilization (%):** This is the amount of space used by the index and cache. A lower space utilization indicates a more efficient use of storage space.

5.4 Discussion on implementation

Figure 2 illustrates the latency results obtained from our proposed scheme and baseline benchmarks. As shown in Fig. 2, it is evident that our proposed scheme surpasses the basic indexing algorithm, LRU caching algorithm, random caching algorithm, and hash-based distribution algorithm in terms of performance. For instance, at 125 requests, our proposed scheme achieved an average latency of 54 ms, while the basic indexing algorithm, LRU caching algorithm, random caching algorithm, and hash-based distribution algorithm exhibited latencies of 104 ms, 80 ms, 69 ms, and 92 ms, respectively. Similarly, with 1000 requests, our proposed scheme maintained a latency of 79 ms,

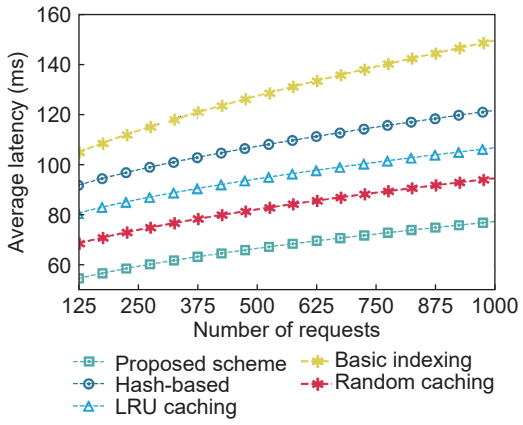

Fig. 2 Average latency comparison of proposed scheme and baseline algorithms.

outperforming the basic indexing algorithm, LRU caching algorithm, random caching algorithm, and hash-based distribution algorithm, which recorded latencies of 150 ms, 100 ms, 94 ms, and 122 ms, respectively. This remarkable performance can be attributed to our scheme's utilization of a distributed indexing system that spreads the index across multiple RSUs, coupled with an efficient caching mechanism for storing frequently accessed data on the RSUs. Additionally, the query optimization algorithm ensures the selection of the most efficient indexing system for retrieving requested data, further minimizing latency.

Figure 3 presents the throughput results of our proposed scheme in comparison to baseline benchmarks. As depicted in Fig. 3, our proposed scheme exhibits superior throughput performance when contrasted with the baseline approaches. For example, with 125 requests, our scheme attains a throughput of approximately 64.10 Mbps, while the hash-based, LRU indexing, random caching, and basic indexing algorithms achieve throughputs of about 46.95 Mbps, 34.29 Mbps, 26.42 Mbps, and 19.93 Mbps, respectively. Similarly, with 1000 requests, our scheme achieves a throughput of approximately 98.41 Mbps, surpassing the hash-based, LRU indexing, random caching, and basic indexing algorithms, which record throughputs of about 78.44 Mbps, 61.89 Mbps, 43.80 Mbps, and 38.76 Mbps, respectively. This notable enhancement in performance is attributed to our proposed scheme's capacity to optimize the indexing of highly dynamic and static data in MR-CAVs, leading to more efficient data storage and retrieval.

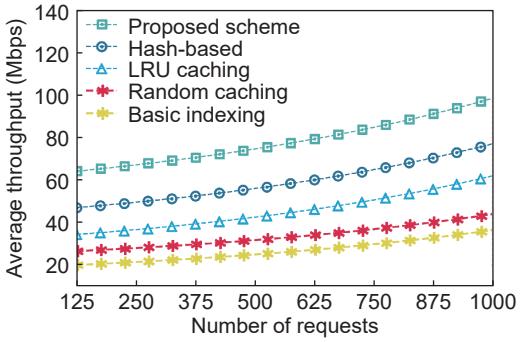

Fig. 3 Average throughput comparison of proposed scheme and baseline algorithms.

5.5 Cache hit rate analysis

Figure 4 presents the results of cache hit rates achieved by our proposed approach and the baseline algorithms. As illustrated in Fig. 4, it is evident that our proposed scheme outperforms the basic indexing algorithm, LRU caching algorithm, random caching algorithm, and hash-based distribution algorithm in terms of cache hit rates. For example, when considering a cache size of 10 GB, our proposed scheme achieves an impressive cache hit rate of 85%, while the basic indexing algorithm, random caching algorithm, LRU caching algorithm, and hash-based distribution algorithm exhibit cache hit rates of 0%, 54%, 58%, and 73%, respectively. This notable improvement can be attributed to the efficiency of our algorithm's indexing technique, which ensures that frequently accessed data are stored in the cache. The superior performance of our proposed scheme is a result of its adept cache management, leading to a higher cache hit rate, reduced latency, and overall system performance enhancement.

5.6 Space utilization analysis

Figure 5 illustrates the results of space utilization achieved by our proposed approach and the baseline algorithms. As observed in Fig. 5, our proposed scheme exhibits superior performance compared to the basic indexing algorithm, LRU caching algorithm, random caching algorithm, and hash-based distribution algorithm. For example, when considering a data size of 30 GB, our scheme attains a space utilization of 40%, while the basic indexing algorithm, LRU caching algorithm, random caching algorithm, and hash-based

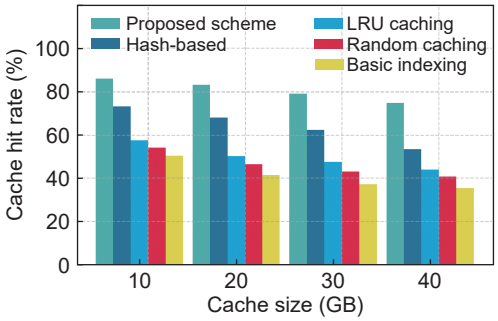

Fig. 4 Cache hit rate comparison of proposed scheme and baseline algorithms.

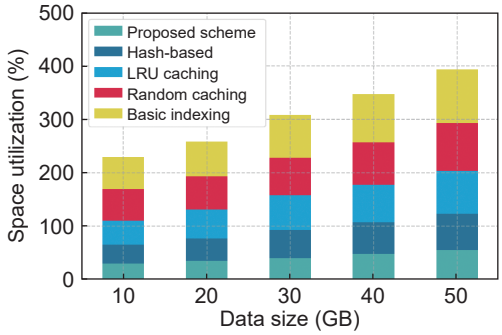

Fig. 5 Space utilization comparison of proposed scheme and baseline algorithms.

distribution algorithm record space utilizations of 80%, 65%, 70%, and 53%, respectively. The reason behind the enhanced performance of our proposed scheme lies in its efficient utilization of RSU cache space, achieved by dynamically adjusting the caching strategy based on data popularity and available space. This approach ensures that frequently accessed data are cached, while less popular data are evicted to accommodate new data, resulting in reduced space usage and improved overall space utilization.

6 Conclusion

This paper proposed an efficient indexing algorithm for highly dynamic and static data in MR-CAVs. Specifically, we formulated our indexing problem as a constrained optimization problem, aiming to maximize the utility function representing the overall performance of our indexing system. This optimization problem encompassed multiple decision variables and constraints, making it mathematically infeasible to solve directly. Therefore, we proposed a heuristic algorithm to address the combinatorial complexity of the problem. Our algorithm efficiently divided data into highly dynamic and static categories and created specialized indexing systems for each category to optimize data retrieval. Next, we designed a query optimization alongside our proposed indexing algorithm to ensure high availability, low latency, and improved performance for efficient data retrieval in vehicular environments. Our proposed algorithm considered the dynamic nature of data and optimized data placement in the cache to reduce latency, increased throughput, and improved cache hit rate and space utilization. Notably, our approach achieved an average reduction in latency ranging from 27% to 49.25%, a 30.75% increase in throughput, a 22.50% improvement in cache hit rate, and a 32% to 50.75% reduction in space utilization. These results highlight the potential of our solution for efficient data management in vehicular networks, particularly in the context of MR-CAVs.

References

[1] M. Abdullah, A. Kiran, and U. Azam, Mobility and content retrieval in vehicular named data network: Challenges and countermeasures, in *Proc. 4th Int. Conf. Advancements in Computational Sciences* (*ICACS*), Lahore, Pakistan, 2023, pp. 1–8.

[2] H. Wang, Z. Wang, D. Chen, Q. Liu, H. Ke, and K. Han, Metamobility: Connecting future mobility with metaverse, arXiv preprint arXiv: 2301.06991, 2023.

[3] H. Yang, H. Pan, and L. Ma, A review on software defined content delivery network: A novel combination of CDN and SDN, *IEEE Access*, vol. 11, pp. 43822–43843, 2023.

[4] E. Bagies, A. Barnawi, S. Mahfoudh, and N. Kumar, Content delivery network for IoT-based fog computing environment, *Comput. Netw.*, vol. 205, p. 108688, 2022.

[5] B. Li and R. Wu, Joint perception data caching and computation offloading in MEC-enabled vehicular networks, *Comput. Commun.*, vol. 199, pp. 139–152, 2023.

[6] R. Song, A. Hegde, N. Senel, A. Knoll, and A. Festag, Edge-aided sensor data sharing in vehicular communication networks, in *Proc. IEEE 95th Vehicular Technology Conf.* (*VTC2022-Spring*), Helsinki, Finland, 2022, pp. 1–7.

[7] F. Zeng, K. Zhang, L. Wu, and J. Wu, Efficient caching in vehicular edge computing based on edge-cloud collaboration, *IEEE Trans. Veh. Technol.*, vol. 72, no. 2, pp. 2468–2481, 2023.

[8] Z. Xue, C. Liu, C. Liao, G. Han, and Z. Sheng, Joint service caching and computation offloading scheme based on deep reinforcement learning in vehicular edge computing systems, *IEEE Trans. Veh. Technol.*, vol. 72, no. 5, pp. 6709–6722, 2023.

[9] R. Greer, N. Deo, A. Rangesh, P. Gunaratne, and M. Trivedi, Safe control transitions: Machine vision based observable readiness index and data-driven takeover time prediction, arXiv preprint arXiv: 2301.05805, 2023.

[10] X. Bi and L. Zhao, Collaborative caching strategy for RL-based content downloading algorithm in clustered vehicular networks, *IEEE Internet Things J.*, vol. 10, no.

11, pp. 9585–9596, 2023.

[11] L. Liu, M. Zhao, M. Yu, M. A. Jan, D. Lan, and A. Taherkordi, Mobility-aware multi-hop task offloading for autonomous driving in vehicular edge computing and networks, *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 2, pp. 2169–2182, 2023.

[12] R. A. Dziyauddin, D. Niyato, N. C. Luong, A. A. A. M. Atan, M. A. M. Izhar, M. H. Azmi, and S. M. Daud, Computation offloading and content caching and delivery in vehicular edge network: A survey, *Comput. Netw.*, vol. 197, p. 108228, 2021.

[13] M. Safwat, A. Elgammal, E. G. AbdAllah, and M. A. Azer, Survey and taxonomy of information-centric vehicular networking security attacks, *Ad Hoc Netw.*, vol. 124, p. 102696, 2022.

[14] Z. Huang and V. Friderikos, Optimal proactive caching for multi-view streaming mobile augmented reality, *Future Internet*, vol. 14, no. 6, p. 166, 2022.

[15] J. A. Khan, C. Westphal, and Y. Ghamri-Doudane, Information-centric fog network for incentivized collaborative caching in the Internet of everything, *IEEE Commun. Mag.*, vol. 57, no. 7, pp. 27–33, 2019.

[16] S. Sukhmani, M. Sadeghi, M. Erol-Kantarci, and A. El Saddik, Edge caching and computing in 5G for mobile AR/VR and tactile Internet, *IEEE Multimed.*, vol. 26, no. 1, pp. 21–30, 2019.

[17] Y. Pei, M. Li, H. Wu, Q. Ye, C. Zhou, S. Hu, and X. Shen, Joint caching and computing resource reservation for edge-assisted location-aware augmented reality, in *Proc. ICC 2023—IEEE Int. Conf. Communications*, Rome, Italy, 2023, pp. 2547–2552.

[18] S. Khriji, Y. Benbelgacem, R. Chéour, D. E. Houssaini, and O. Kanoun, Design and implementation of a cloud-based event-driven architecture for real-time data processing in wireless sensor networks, *J. Supercomput.*, vol. 78, no. 3, pp. 3374–3401, 2022.

[19] W. Fan, Y. Su, J. Liu, S. Li, W. Huang, F. Wu, and Y. Liu, Joint task offloading and resource allocation for vehicular edge computing based on V2I and V2V modes, *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 4, pp. 4277–4292, 2023.

[20] C. Chen, C. Wang, L. Cong, M. Xiao, and Q. Pei, A V2V emergent message dissemination scheme for 6G-oriented vehicular networks, *Chin. J. Elect.*, vol. 32, no. 6, pp. 1179–1191, 2023.

[21] D. M. Doe, D. Chen, K. Han, Y. Dai, J. Xie, and Z. Han, Real-time search-driven content delivery in vehicular networks for AR/VR-enabled autonomous vehicles, in *Proc. IEEE/CIC Int. Conf. Communications in China* (*ICCC*), Dalian, China, 2023, pp. 1–6.

[22] I. Kemouguette, Z. Kouahla, A. E. Benrazek, B. Farou, and H. Seridi, Cost-effective space partitioning approach for IoT data indexing and retrieval, in *Proc. Int. Conf. Networking and Advanced Systems* (*ICNAS*), Annaba, Algeria, 2021, pp. 1–6.

[23] L. Zhang, M. Peng, W. Wang, Z. Jin, Y. Su, and H. Chen, Secure and efficient data storage and sharing scheme for blockchain-based mobile-edge computing, *Trans. Emerg. Telecommun. Technol.*, vol. 32, no. 10, p. e4315, 2021.

[24] Z. Su, Y. Hui, Q. Xu, T. Yang, J. Liu, and Y. Jia, An edge caching scheme to distribute content in vehicular networks, *IEEE Trans. Veh. Technol.*, vol. 67, no. 6, pp. 5346–5356, 2018.

[25] Y. Chen, Y. Sun, B. Yang, and T. Taleb, Joint caching and computing service placement for edge-enabled IoT based on deep reinforcement learning, *IEEE Internet Things J.*, vol. 9, no. 19, pp. 19501–19514, 2022.

[26] C. Yang, P. Jiang, and L. Zhu, Accelerating decentralized and partial-privacy data access for VANET via online/offline functional encryption, *IEEE Trans. Veh. Technol.*, vol. 71, no. 8, pp. 8944–8954, 2022.

[27] H. Noori and M. Valkama, Impact of VANET-based V2X communication using IEEE 802.11p on reducing vehicles traveling time in realistic large scale urban area, in *Proc. Int. Conf. Connected Vehicles and Expo* (*ICCVE*), Las Vegas, NV, USA, 2013, pp. 654–661.

Daniel Mawunyo Doe received the BS degree in computer engineering from Kwame Nkrumah University of Science and Technology, Kumasi, Ghana, and the MS degree in computer science and engineering from University of Electronic Science and Technology of China (UESTC), China. He is currently pursuing the PhD degree at the Department of Electrical and Computer Engineering, University of Houston, USA. He is a member of IEEE. His general research interests include game theory, blockchains, federated learning, wireless networks, big data, and cloud computing.

Dawei Chen received the BS degree from Huazhong University of Science and Technology, Wuhan, China, in 2015, and the PhD degree from University of Houston, Houston, TX, USA, in 2021. After this, he joined the InfoTech Labs, Toyota Motor North America Research and Development, USA, where he is a principal researcher. He is a member of IEEE. His research interests include edge/cloud computing, federated learning/ analytics, connected vehicles, and wireless networks.

Kyungtae Han received the PhD degree in electrical and computer engineering from The University of Texas at Austin, Austin, TX, USA, in 2006. He is currently a senior principal scientist at the InfoTech Labs, Toyota Motor North America Research and Development, USA. Prior to joining Toyota, he was a research scientist with Intel Labs, Santa Clara, CA, USA, and a director of Locix Inc., San Bruno, CA, USA. He is a senior member of IEEE. His research interests include cyber–physical systems, connected and automated vehicle techniques, and intelligent transportation systems.

Haoxin Wang received the PhD degree in electrical and computer engineering from The University of North Carolina at Charlotte, USA, in 2020, and the BS degree in control science and engineering from Harbin Institute of Technology, China, in 2015. From 2020 to 2022, he was a research scientist at the InfoTech Labs, Toyota Motor North America Research and Development, USA. He is currently an assistant professor at the Department of Computer Science, Georgia State University, USA, and leads the Advanced Mobility & Augmented Intelligence (AMAI) Lab. He is a member of IEEE. His current research interests include mobile MR, autonomous driving, digital twins, and edge intelligence.

Jiang Xie received the BEng degree in electrical and computer engineering from Tsinghua University, Beijing, China, the MPhil degree in electrical and computer engineering from Hong Kong University of Science and Technology, China, and the MS and PhD degrees in electrical and computer engineering from the Georgia Institute of Technology, USA. She joined the Department of Electrical and Computer Engineering, University of North Carolina at Charlotte (UNC-Charlotte), USA as an assistant professor in August 2004, where she is currently a full professor. She is a fellow of IEEE. Her current research interests include resource and mobility management in wireless networks, mobile computing, Internet of Things, and cloud/edge computing. She received the U.S. National Science Foundation NSF Faculty Early Career Development (CAREER) Award in 2010, the Best Paper Award from IEEE Global Communications Conference in 2017, the Best Paper Award from IEEE/WIC/ACM International Conference on Intelligent Agent Technology in 2010, and the Graduate Teaching Excellence Award from the College of Engineering at UNC-Charlotte in 2007. She is on the editorial boards of *IEEE Transactions on Wireless Networking*, *IEEE Transactions on Sustainable Computing*, and *Journal of Network and Computer Applications* (Elsevier).

Zhu Han received the BS degree in electronic engineering from Tsinghua University, China in 1997, and the MS and PhD degrees in electrical and computer engineering from University of Maryland, College Park, USA in 1999 and 2003, respectively. From 2000 to 2002, he was an R&D Engineer of Jones, Duck and Straus/Sinclair Uniphase (JDSU), Germantown, Maryland, USA. From 2003 to 2006, he was a research associate at University of Maryland, USA. From 2006 to 2008, he was an assistant professor at Boise State University, Idaho, USA. Currently, he is a John and Rebecca Moores Professor at the Department of Electrical and Computer Engineering as well as at the Department of Computer Science, University of Houston, Texas, USA. His main research targets on the novel game-theory related concepts critical to enabling efficient and distributive use of wireless networks with limited resources. His other research interests include wireless resource allocation and management, wireless communications and networking, quantum computing, data science, smart grid, and security and privacy. He received an NSF Career Award in 2010, the Fred W. Ellersick Prize of the IEEE Communication Society in 2011, the EURASIP Best Paper Award for the *Journal on Advances in Signal Processing* in 2015, IEEE Leonard G. Abraham Prize in the field of Communications Systems (best paper award in IEEE JSAC) in 2016, and several best paper awards in IEEE conferences. He was an IEEE Communications Society Distinguished Lecturer from 2015−2018. He has been a fellow of AAAS since 2019, and an ACM distinguished member since 2019. He has been a 1% highly cited researcher since 2017 according to Web of Science. He is also the winner of the 2021 IEEE Kiyo Tomiyasu Award, for outstanding early to mid-career contributions to technologies holding the promise of innovative applications, with the following citation: “for contributions to game theory and distributed management of autonomous communication networks”. He is a fellow of IEEE.